

# IV RTOS - Semaphores

Contreras R Orlando<sup>1,2\*</sup> and Lara H Kevin<sup>1,2\*</sup>

<sup>3</sup>Department of Electronic Systems, UAA, Av. Universidad 940,  
Aguascalientes, 20131, Aguascalientes, México.

\*Corresponding author(s). E-mail(s): [{al348390,al464376}@edu.uaa.mx](mailto:{al348390,al464376}@edu.uaa.mx);

## Abstract

The objective of this lab practice is handle multiple semaphore to had a sequential behavioral with Tasks on RTOS Framework. The system employs a multi-task architecture with five concurrent tasks managing three LED states (red, green, yellow) through binary semaphore synchronization and queue-based inter-task communication

**Keywords:** ARM, C, Priorities, Queues, RTOS, Semaphores, Task

## 1 Introduction

In this practice we will handle a semaphore to had a sequence in task, this to avoid happening critical task at the same time and control the running cycle of each task in this case we will create a 3 color LED semaphore using the FreeRTOS binary semaphore function to implement the correct use of semaphores, in addition we use a queue to handle a button task to switch the mode to blink the three LEDs at the same time, if the same button is pressed the semaphore would operate normal.

## 2 Objectives

- Practice creating and synchronizing FreeRTOS tasks.
- Use binary semaphores to control execution order among tasks.
- Understand priorities, `vTaskDelay()` semantics and inter-task communication.
- Correct use of queues to handle values between tasks.
- Defining priorities and semaphores to avoid timing errors.
- Correct use of GPIO.

## 3 Method

### 3.1 Pseudo-code

Here is the pseudo-code that represents the algorithm, it is also included the configuration as masks or

---

**Algorithm 1** Main System Initialization

---

**Input:** System initialized

**Output:** Running RTOS with LED control system

```
1 Initialize system configureRCC() // Configure clock for GPIO ports A and E
2 configureGPIO()                // Configure GPIO modes for LEDs and button
3 createTasks()                  // Create queues, semaphores, and tasks
4 startScheduler()               // Start FreeRTOS scheduler
```

---

---

**Algorithm 2** Task and Resource Creation

---

**Input:** System resources available

**Output:** Tasks, queues, and semaphores created

```
5 xCONTROLQueue ← createQueue(1, sizeof(uint8)) // Control command queue
6 xSEMQueue ← createQueue(1, sizeof(uint8))      // Semaphore control queue
7 SemaphoreRED ← createBinarySemaphore()         // Red LED semaphore
8 SemaphoreGREEN ← createBinarySemaphore()       // Green LED semaphore
9 SemaphoreYELLOW ← createBinarySemaphore()      // Yellow LED semaphore
10 SemaphoreBLINK ← createBinarySemaphore()       // Blink mode semaphore
11 giveSemaphore(SemaphoreRED)                   // Start with red LED active
12 createTask(vTaskButton, "Task Button", 100, NULL, 3, ButtonHandle)
   createTask(vTaskBlink, "Task Blink", 100, NULL, 2, BlinkHandle)
   createTask(vTaskLEDR, "Task LEDR", 100, NULL, 1, LEDRHandle)
   createTask(vTaskLEDG, "Task LEDG", 100, NULL, 1, LEDGHandle)
   createTask(vTaskLEDY, "Task LEDY", 100, NULL, 1, LEDYHandle)
```

---

---

**Algorithm 3** Red LED Task

---

**Input:** *SemaphoreRED* available

**Output:** Red LED sequence executed

```
13 while true do
14   takeSemaphore(SemaphoreRED, portMAX_DELAY) // Wait for red
   semaphore
15   GPIOA.ODR ← (1 << RED)                    // Turn on red LED
16   delay(pdMS_TO_TICKS(5000))                // Wait 5 seconds
17   GPIOA.ODR ← (0 << RED)                    // Turn off red LED
18   if (receiveQueue(xSEMQueue, recievedQueue, pdMS_TO_TICKS(20))) &
   (recievedQueue = 255) then
19     giveSemaphore(SemaphoreBLINK)            // Enter blink mode
20   else
21     giveSemaphore(SemaphoreGREEN)           // Pass to green LED
```

---

---

**Algorithm 4** Green LED Task

---

**Input:** *SemaphoreGREEN* available**Output:** Green LED sequence executed

```
22 while true do
23   takeSemaphore(SemaphoreGREEN, portMAX_DELAY)      // Wait for
   green semaphore
24   GPIOA.ODR  $\leftarrow$  (1  $\ll$  GREEN)                    // Turn on green LED
25   delay(pdMS_TO_TICKS(3000))                          // Wait 3 seconds
26   GPIOA.ODR  $\leftarrow$  (0  $\ll$  GREEN)                    // Turn off green LED
27   if (receiveQueue(xSEMQueue, recievedQueue, pdMS_TO_TICKS(10)))  $\wedge$ 
   (recievedQueue = 255) then
28     | giveSemaphore(SemaphoreBLINK)                  // Enter blink mode
29   else
30     | giveSemaphore(SemaphoreYELLOW)                // Pass to yellow LED
```

---

---

**Algorithm 5** Yellow LED Task

---

**Input:** *SemaphoreYELLOW* available**Output:** Yellow LED blinking sequence executed

```
31 while true do
32   takeSemaphore(SemaphoreYELLOW, portMAX_DELAY)    // Wait for
   yellow semaphore
33   for i  $\leftarrow$  0 to 8 do
34     | GPIOA.ODR  $\leftarrow$  (1  $\ll$  YELLOW)                // Turn on yellow LED
35     | delay(pdMS_TO_TICKS(250))                      // Wait 250ms
36     | GPIOA.ODR  $\leftarrow$  (0  $\ll$  YELLOW)                // Turn off yellow LED
37     | delay(pdMS_TO_TICKS(250))                      // Wait 250ms
38   if (receiveQueue(xSEMQueue, recievedQueue, pdMS_TO_TICKS(20)))  $\wedge$ 
   (recievedQueue = 255) then
39     | giveSemaphore(SemaphoreBLINK)                  // Enter blink mode
40   else
41     | giveSemaphore(SemaphoreRED)                   // Return to red LED
```

---

---

**Algorithm 6** Blink Mode Task

---

**Input:** *SemaphoreBLINK* available**Output:** All LEDs blinking pattern executed

```
42 while true do
43   takeSemaphore(SemaphoreBLINK, portMAX_DELAY) // Wait for blink
    semaphore
44   GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\wedge \sim(0xFF \ll 0)$  // Clear all LEDs
45   receiveQueue(xCONTROLQueue, sendstate, pdMS_TO_TICKS(10)) while
    sendstate = 0xFF do
46     GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\oplus (1 \ll \text{YELLOW})$  // Toggle yellow LED
47     GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\oplus (1 \ll \text{RED})$  // Toggle red LED
48     GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\oplus (1 \ll \text{GREEN})$  // Toggle green LED
49     delay(pdMS_TO_TICKS(500)) // Wait 500ms
50     if (receiveQueue(xCONTROLQueue, sendstate, pdMS_TO_TICKS(10)))
        then
51        $\lfloor$  sendstate  $\leftarrow$  5 // Exit blink mode
52   GPIOA.ODR  $\leftarrow$  GPIOA.ODR  $\wedge \sim(0xFF \ll 0)$  // Clear all LEDs
53   resetQueue(xSEMQueue) // Reset semaphore queue
54   resetQueue(xCONTROLQueue) // Reset control queue
55   giveSemaphore(SemaphoreRED) // Return to normal operation
```

---

---

**Algorithm 7** Button Task

---

**Input:** Button input on PE8**Output:** Button presses detected and commands sent

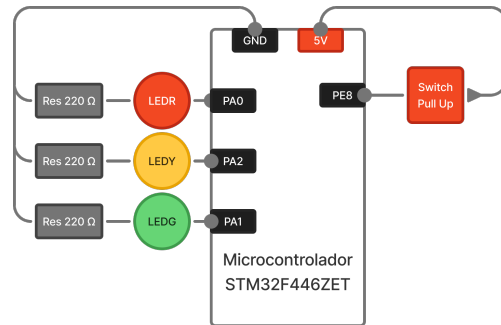
```
56 lastState  $\leftarrow$  1 cont  $\leftarrow$  0 while true do
57   if  $\neg(\text{GPIOE.IDR} \wedge (1 \ll 8))$  then
58      $\lfloor$  actualState  $\leftarrow$  0 // Button pressed
59   else
60      $\lfloor$  actualState  $\leftarrow$  1 // Button released
61   if (lastState = 1)  $\wedge$  (actualState = 0) then
62     delay(pdMS_TO_TICKS(20)) // Debounce delay
63     sendstate  $\leftarrow \sim \text{actualState}$  // Set to 0xFF
64     sendQueue(xSEMQueue, sendstate, portMAX_DELAY)
        sendQueue(xCONTROLQueue, sendstate, portMAX_DELAY)
65     if cont = 1 then
66       sendQueue(xCONTROLQueue, sendstate, portMAX_DELAY) cont  $\leftarrow$ 
        0 // Reset counter
67     else
68        $\lfloor$  cont  $\leftarrow$  cont + 1 // Increment counter
69    $\lfloor$  lastState  $\leftarrow$  actualState delay(pdMS_TO_TICKS(50)) // Polling interval
```

---

## 4 Results and Discussion

### 4.1 Wiring Diagram

Here is our circuit implementation; we configure the button as pull up as shown in [Fig 1]



**Fig. 1:** Wiring Diagram

**Q1:** Why use three semaphores rather than a single one? Pros/cons.

The use of three semaphores is useful to switch between task execution cycle, in three LEDs the semaphores can be used to change which led is ON and which LED is the next to turn ON. However, the use of semaphores could be confusing and impractical when yo have a lot of task to control.

**Q2:** What happens if ButtonTask had low priority and the button is pressed?

If the button task had a low priority and another task with higher priority is running at the same time, is probably that the button task doesn't have registered the button press.

**Q3:** How to reduce latency in mode switching if `vTaskDelay()` is large?

To reduce the latency we could implement interruptions to swtich the mode instead of polling the button

**Q4:** Improvements for power and responsiveness (timers, direct-notify, PWM dimming).

To implement timing imprpovements we could use the Hardware timers (TIM1-TIM5) should replace software delays for precise timing control while reducing CPU overhead.

## 5 Conclusion

In this practice, we could observe the functionality of Semaphores to avoid multitasking on critical task, this is a very useful way to have an order of an algorithm because we could seize the concurrency of FreeRTOS with a clean flow of task, merging the both ways of doing a process.